Ensembles and random forests

Forest CoverType · many models, and what averaging can and cannot do

LECTURE 7 | GÉRON CH. 6

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

The previous lecture built a decision tree on this same dataset — 60,000 patches of forest, 54 cartographic columns, seven species, split 48,000 / 12,000. CART chose each split to minimise the size-weighted impurity of the two children, and the brief's eight conditions per prediction fixed the depth at 8.

Forest CoverType	Test accuracy	Conditions per justification
Always "Lodgepole Pine"	48.8%	0
Depth-8 tree, minimum leaf 20	73.0%	7.97
Unconstrained tree	82.6%	17.88

Nothing on that table is wrong. It is also not the whole story, and the missing part is what today repairs.

The one result to carry forward

MEASURED IN THIS LECTURE'S NOTEBOOK, BEFORE ANYTHING IS AVERAGED

Refit the same depth-8 tree, with the same hyperparameters and the same seed, on twenty different 90% subsamples of the same training rows. Test accuracy moves by a standard deviation of **0.25 points** — and **9.1%** of the individual predictions change. Only **71.4%** of test patches get the same answer from all twenty.

A stable metric, an unstable model. In the language of Lecture 5's decomposition, this is **variance**: sensitivity of the fitted function to the particular training sample.

✓ The whole of today follows from one question: what does *averaging* do to variance?

Today

1. Why many models, and why trees are the ones to average
2. **The mathematics** — the variance of an average of n correlated predictors, and the floor that n cannot reach
3. **The method** — voting, bagging, out-of-bag evaluation, random forests and extra-trees; ρ and the accuracy measured; what averaging costs; feature importance with a control
4. Boosting, the histogram-based implementations, stacking, and which to reach for

Open Lecture 7 in Colab

The idea

Ask a difficult question of a thousand random people and aggregate their answers. The aggregate is often better than an expert's. The same is true of models, and it has a name: an **ensemble**.

> *A group of weak learners can form a strong learner — provided there are enough of them and they are sufficiently diverse.*

"Sufficiently diverse" is doing all the work in that sentence, and it is the informal name of a quantity we are about to define. Before long it will have a symbol, a formula and a measured value on this dataset.

Why this lecture follows the one on trees

- Averaging attacks **variance**, and trees are the high-variance model we have just measured: 9.1% of predictions move when 10% of the rows do
- A tree is cheap to fit, so two hundred of them are affordable
- An unconstrained tree has **low bias** — and averaging does nothing whatever to bias, so the members had better not have any to spare

> That third point is why the ensembles in this lecture use *unconstrained* members. It looks reckless after a lecture spent limiting depth, and by the end of the mathematics it will look obvious.

The question

Forget trees for a moment.

You have n predictors of the same quantity. Each has variance σ^2. You average them.

$$\bar{f} = \frac{1}{n}\sum_{t=1}^{n} f_t$$

What is $\mathrm{Var}(\bar{f})$?

Start with two, then take n

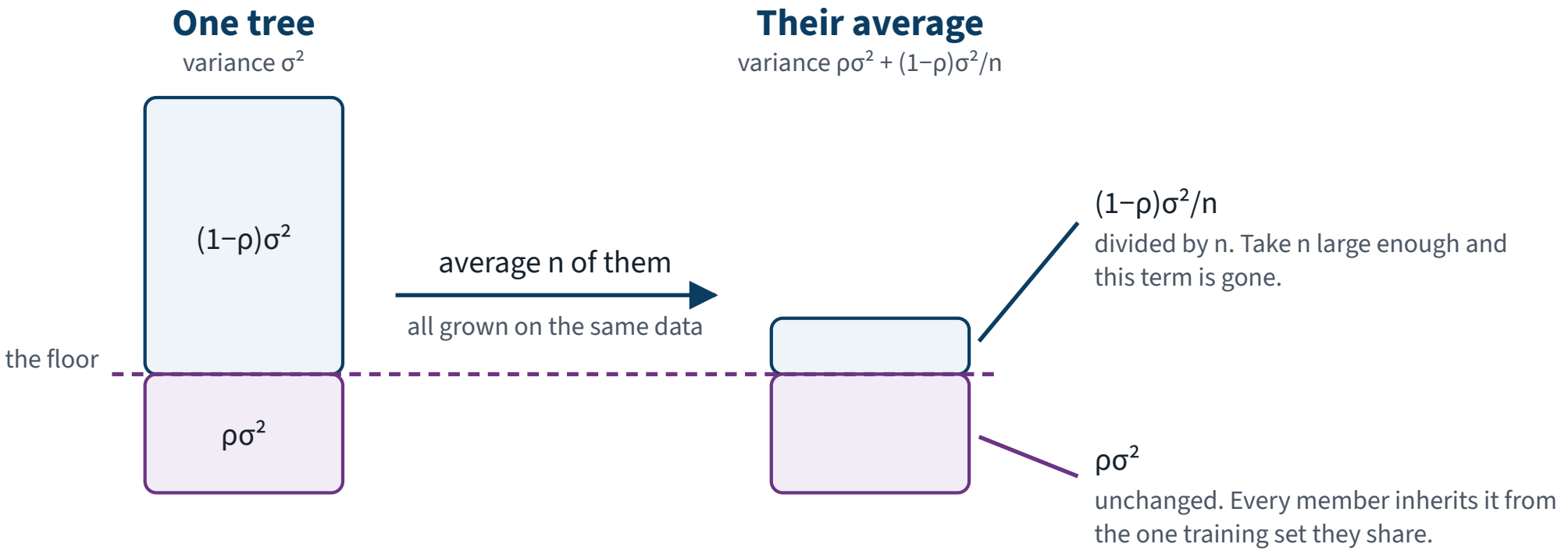
With $\mathrm{Var}(f_1) = \mathrm{Var}(f_2) = \sigma^2$ and $\mathrm{Corr}(f_1, f_2) = \rho$:

$$\mathrm{Var}\left(\frac{f_1+f_2}{2}\right) = \frac{1}{4}\left[\sigma^2 + \sigma^2 + 2\rho\sigma^2\right] = \frac{\sigma^2(1+\rho)}{2}$$

$\rho = 0$ halves the variance; $\rho = 1$ leaves it at σ^2, and averaging achieves nothing. For n identically distributed predictors with a common pairwise correlation ρ, the same expansion — n variances and $n(n-1)$ covariances $\rho\sigma^2$, over n^2 — gives

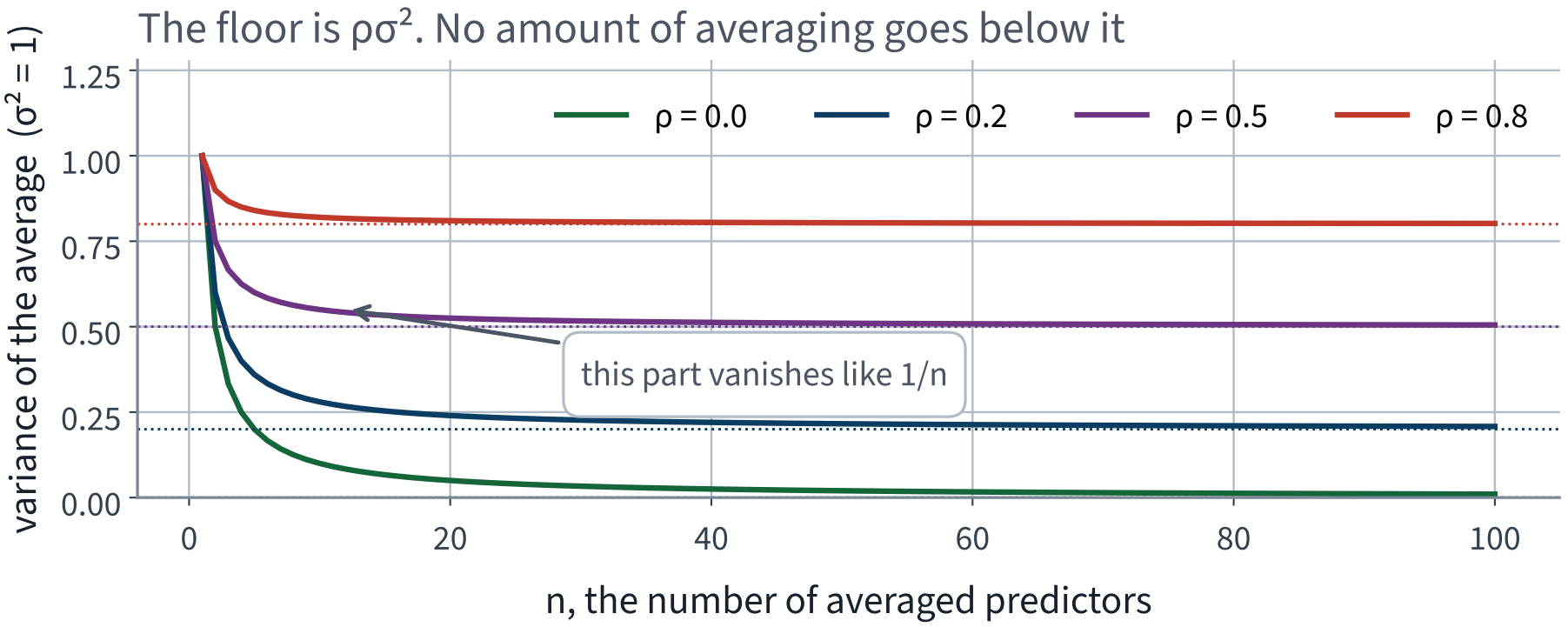
$$\mathrm{Var}(\bar{f}) \;=\; \rho\sigma^2 \;+\; \frac{(1-\rho)\,\sigma^2}{n}$$

The formula, in one picture



Everything above the dashed line can be averaged away. Nothing below it can.

The same formula, plotted



Four curves, four floors. The floor is set entirely by ρ — which is why every ensemble variant you are about to meet is an attempt to lower it.

What this says before we build anything

1. Averaging many models reduces variance, but only the part they do not share
2. The return on more members collapses quickly. There is little point past the point where $(1-\rho)\sigma^2/n \ll \rho\sigma^2$
3. Therefore the way to improve an ensemble is not more members. It is **less correlated** members
4. And anything that lowers ρ by making individual members worse is a trade, not a free lunch: σ^2 appears in *both* terms, so it can rise while ρ falls

✓ Point 4 is the one that gets forgotten, and it is the one we will measure — and be embarrassed by — later today.

Voting — the simplest ensemble

Combine *different* algorithms fitted to the same data.

```
from sklearn.ensemble import VotingClassifier

vote = VotingClassifier(
    [("tree",   DecisionTreeClassifier(random_state=42)),
     ("logreg", LogisticRegression(max_iter=1000)),
     ("knn",    KNeighborsClassifier(n_neighbors=5))], voting="soft")
```

- `voting="hard"` — majority of the predicted labels
- `voting="soft"` — average of the predicted probabilities; usually better, because a confident member outweighs several hesitant ones

Different algorithms make different kinds of errors, which is the informal way of saying ρ is small. The formula applies unchanged.

Bagging

Bootstrap **agg**regat**ing**. Draw m rows from the training set *with replacement*, fit a tree, and repeat n times.

- Each bootstrap sample has the same size as the training set, but about **63%** of its distinct rows — the rest are duplicates
- Aggregate by majority vote for classification, by the mean for regression
- Each member is individually worse than one tree fitted to the whole set. The ensemble is better — that is the formula, not a paradox

Pasting is the same thing without replacement. It produces less diverse members, so bagging usually wins — but it is a cheap thing to cross-validate rather than assume.

Bagging, in code

```
from sklearn.ensemble import BaggingClassifier

bag = BaggingClassifier(
    DecisionTreeClassifier(random_state=42),   # unconstrained members
    n_estimators=200, bootstrap=True, oob_score=True,
    random_state=42, n_jobs=-1)

bag.fit(X_train, y_train)
print(f"out-of-bag {bag.oob_score_:.1%}")
```

The members are **unconstrained** trees. We want each one to have low bias, because averaging cannot remove bias — and we are about to average the variance away, so there is no reason to restrict their depth.

A validation set you get for nothing

Each member never sees about 37% of the rows — its **out-of-bag** instances. Score each member on its own out-of-bag rows and you have a generalisation estimate with no validation split and no extra fit.

> **WHY 37%**
>
> A given row is missed by one draw with probability $1 - 1/m$, so by all m draws with probability $\left(1 - \frac{1}{m}\right)^m \to e^{-1} \approx 0.368$ as m grows. Hence about 63% distinct rows in, 37% held out.

Cheap, and it does not replace the test set. It replaces the *validation* set. If out-of-bag comes out much better than the test score, something in your pipeline saw the out-of-bag rows anyway.

Reading an out-of-bag score

- It is an average over members, each scored on a *different* 37% of the rows — so it estimates the generalisation of one member's procedure, not of the ensemble you ship
- It is available only where the members see bootstrap samples. With `bootstrap=False` — the extra-trees default — there are no out-of-bag rows and `oob_score=True` raises
- Use it to compare configurations cheaply, then confirm the one you pick with cross-validation or the test set

> **WHEN IT BITES**
>
> If the out-of-bag score comes out much *better* than the test score, something in the pipeline saw the out-of-bag rows anyway — a scaler fitted outside the ensemble, or a feature built from the whole frame. The gap is a leakage detector you get for free.

Random forests

Bagging, plus one more source of difference: at **every node of every tree**, only a random subset of the features is considered for the split.

```
from sklearn.ensemble import RandomForestClassifier

rnd = RandomForestClassifier(n_estimators=200, max_features="sqrt",
                             random_state=42, n_jobs=-1).fit(X_train, y_train)
```

`max_features="sqrt"` is the classification default: $\lfloor \sqrt{54} \rfloor = 7$ of the 54 columns, drawn afresh at each node. A tree with 200 nodes therefore draws 200 different subsets.

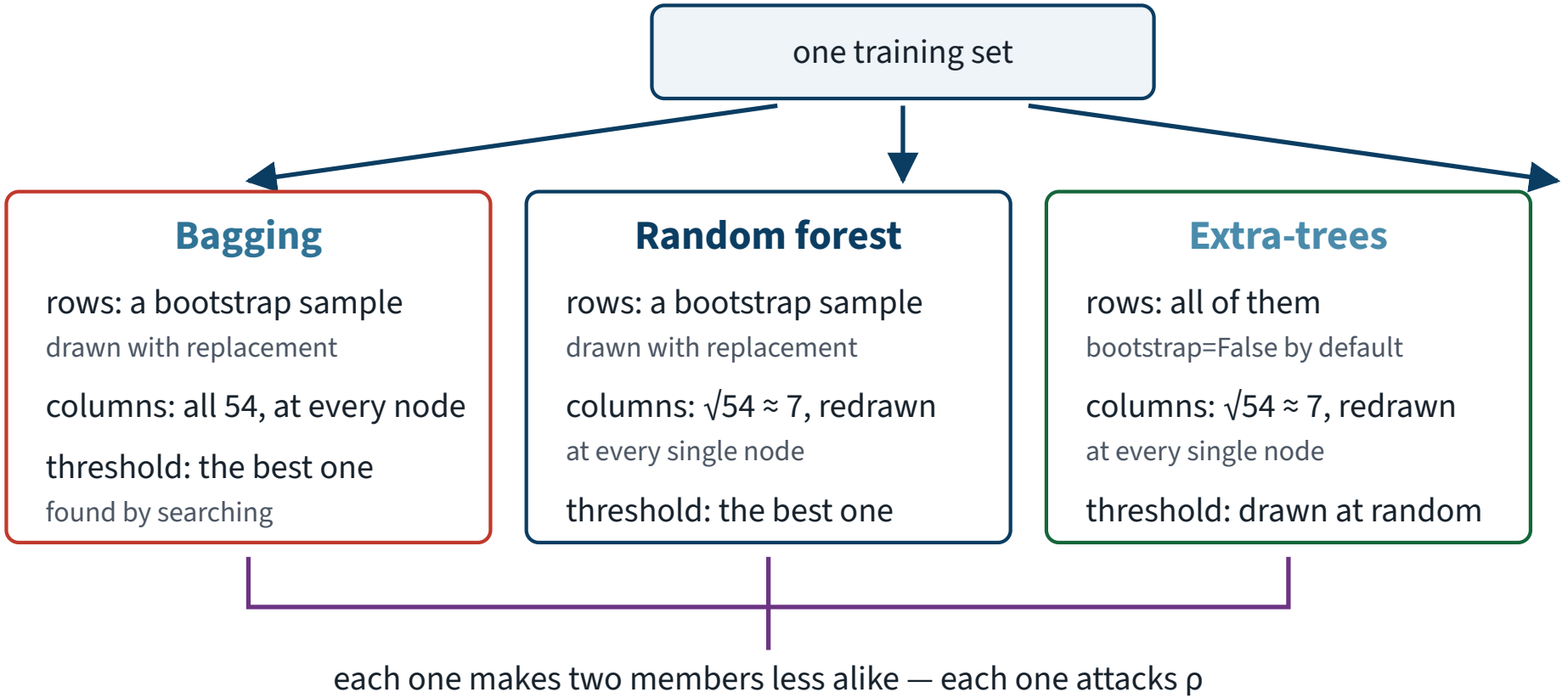
Extra-trees

One further step. Having chosen the candidate features, do not search for the best threshold — **draw one at random** for each candidate and keep the best of those.

- Much faster: the threshold search is the expensive part of training a tree
- On bootstrap samples, each member comes out a little worse than a forest's (72.8% against 73.6%)
- Scikit-Learn's default is `bootstrap=False` — the random thresholds *replace* the bootstrap rather than joining it

✗ That default is a decision somebody else made, it is not what most descriptions of extra-trees imply, and we are going to measure what it costs rather than argue about it.

What each variant randomises



Rows, columns, thresholds. Three knobs, and all three are turned for the same reason.

The claim this lecture is making

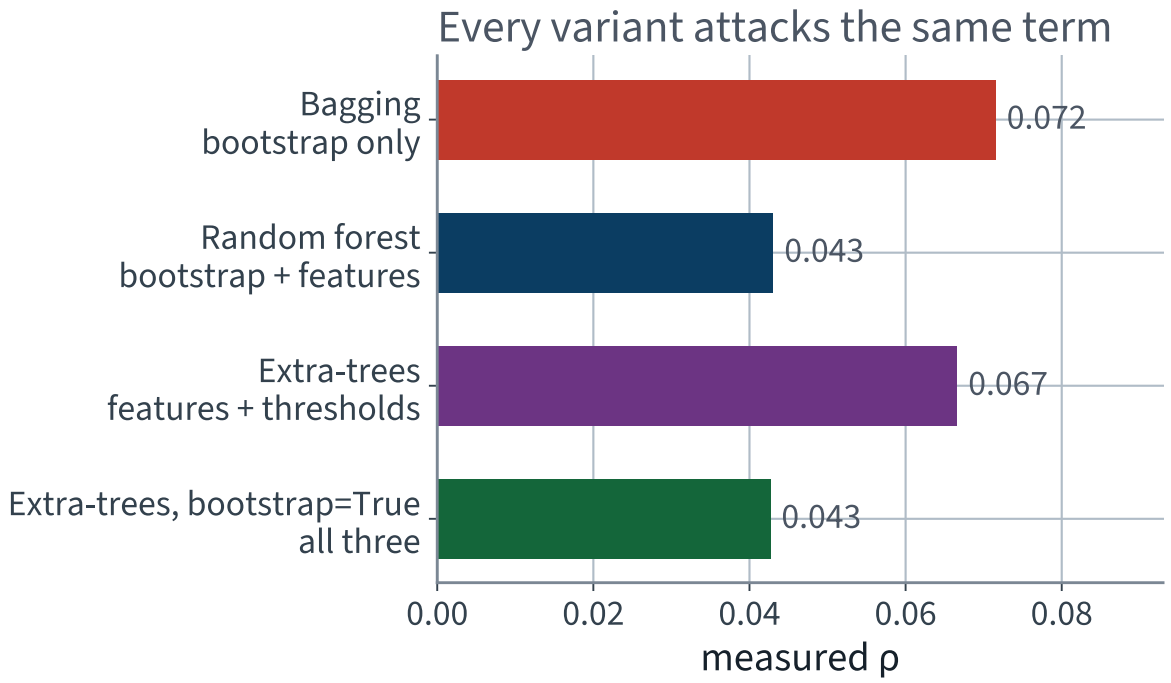
ONE FORMULA, THREE ALGORITHMS

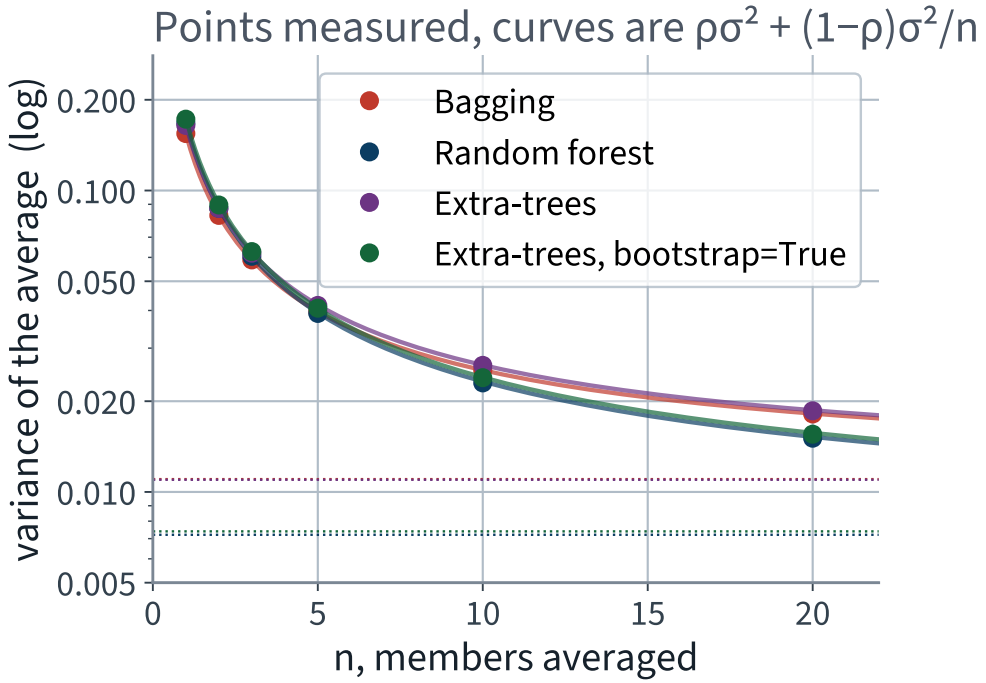
Bagging randomises the **rows**. Random forests also randomise the **columns**. Extra-trees also randomise the **thresholds**. All three exist to reduce one quantity: ρ, the correlation term in $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/n$.

That is the payoff of the derivation, and it is the thing to remember if you remember one thing from this lecture.

✗ It is also a claim, and this course does not accept claims. Measure it.

ρ, measured on four ensembles





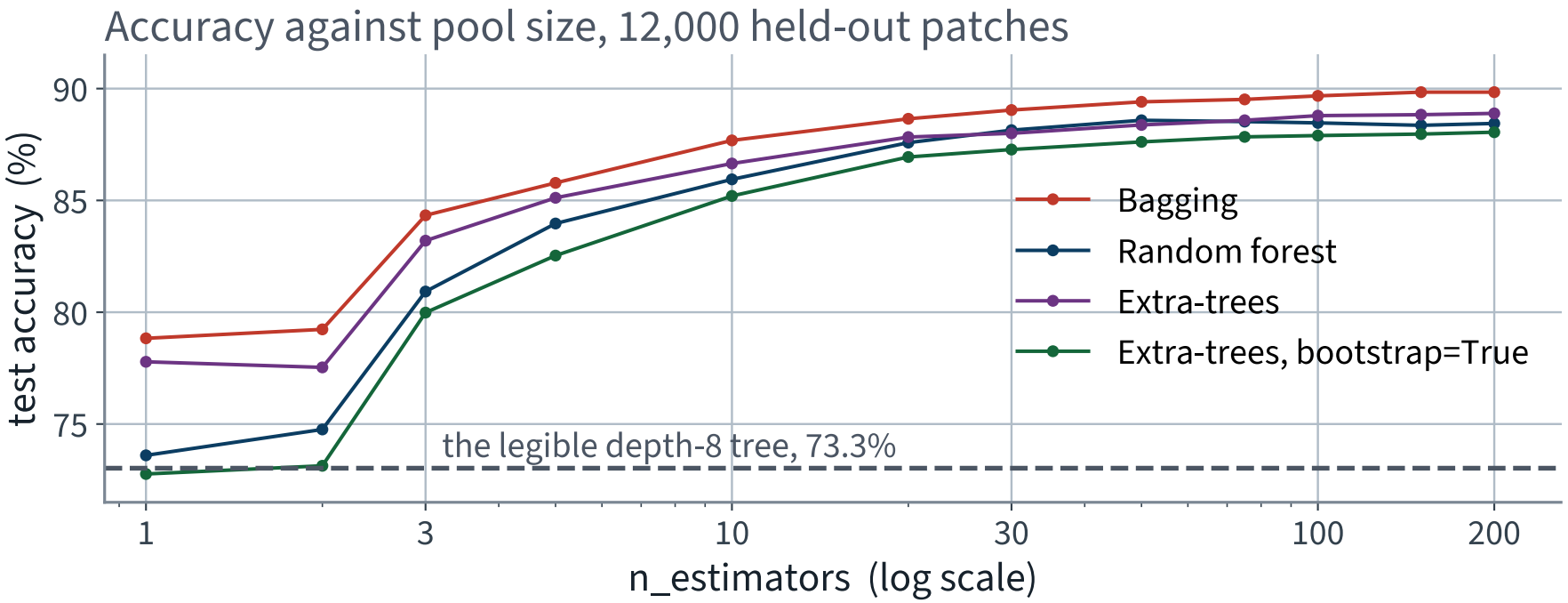
Measured across 20 disjoint training sets of 20,000 rows, so that the shared part of the variance has something to come from. The points are measured and the curves are $\rho\sigma^2 + (1-\rho)\sigma^2/n$; nothing was fitted to make them line up.

And what did that do to the accuracy?

200 members, 48,000 training patches	10 members	200 members
Bagging	87.7%	**89.8% ✓**
Extra-trees	86.7%	88.9%
Random forest	85.9%	88.4%
Extra-trees, `bootstrap=True`	85.2%	88.0%
The legible depth-8 tree	—	73.0%
Always “Lodgepole Pine”	—	48.8%

✓ Every ensemble beats the constrained tree by at least fifteen points, and the *unconstrained* tree by at least five. The last 190 members buy about two.

How many members do you actually need?



Most of the gain arrives by thirty members, exactly as $(1 - \rho)\sigma^2/n$ says it should.

Now look at that ranking again

Ensemble	ρ	One member's accuracy	Ensemble accuracy
Bagging	**✗ 0.072**	**78.8% ✓**	**89.8% ✓**
Extra-trees	0.067	77.8%	88.9%
Random forest	**0.043 ✓**	73.6%	88.4%
Extra-trees, `bootstrap=True`	**0.043 ✓**	**✗ 72.8%**	**✗ 88.0%**

✗ The ensemble with the *highest* ρ is the most accurate.

✓ Point 4 of the formula's reading, measured: every mechanism that lowers ρ does it by handicapping the members, σ^2 sits in both terms, and the formula says nothing about bias.

Now the bill

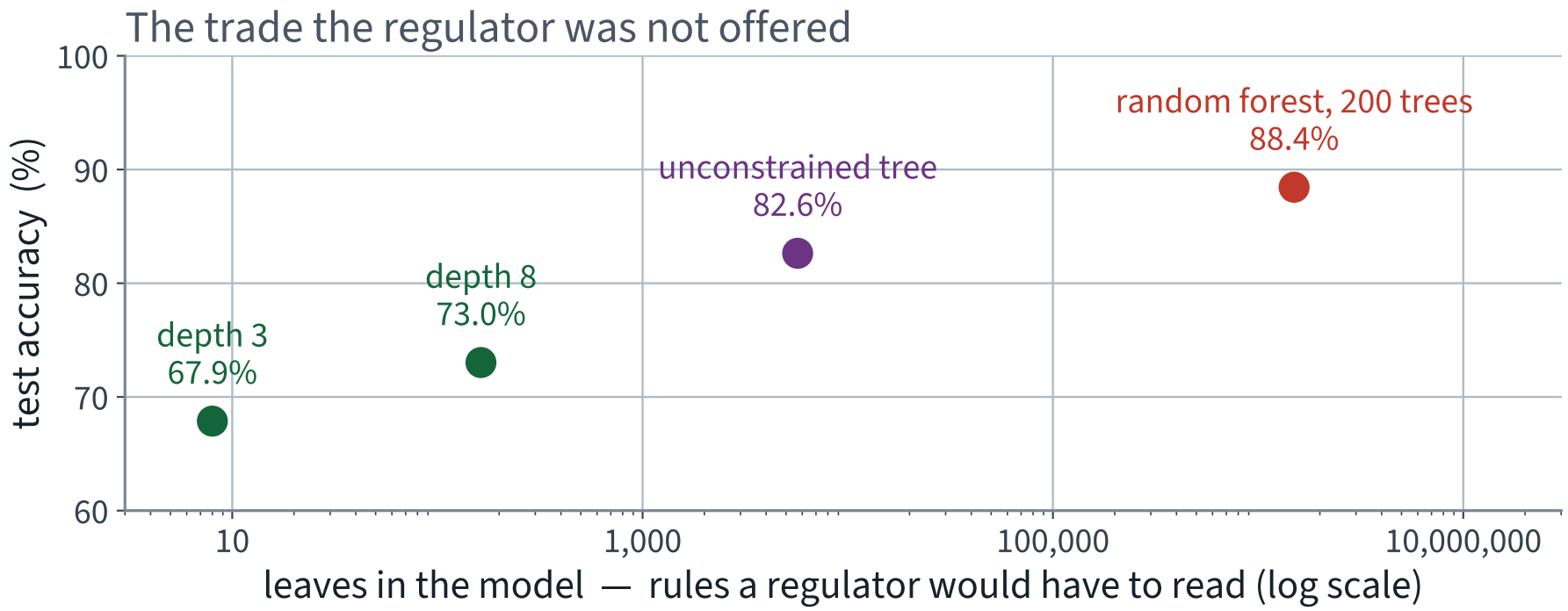
We removed the variance. Ask what happened to the thing the agency asked for.

```
total = sum(t.get_n_leaves() for t in rnd.estimators_)
print(f"{total:,} leaves across the 200 trees")
```

```
1,497,919 leaves across the 200 trees
```

✗ The justification for one prediction is now 200 decision paths and a vote.

The trade, drawn to scale



✗ Fifteen points of accuracy, and four orders of magnitude more model to read.

What the vote share is, and is not

A single tree's `predict_proba` returns the class proportions of one leaf: piecewise constant, identical for every patch reaching that leaf, and often built on a handful of training rows.

An ensemble averages 200 such vectors, each from a different leaf of a different tree. The result varies smoothly across the feature space and is **better calibrated** — one thing the agency asked for that averaging improves rather than destroys.

> **WHEN IT BITES**
>
> Better calibrated is not calibrated. A forest's vote share is systematically pulled towards the middle, because unanimity is rare. If a downstream decision uses the number as a probability, check it against observed frequencies before believing it.

Feature importance, as the library offers it

```
imp = pd.Series(rnd.feature_importances_, index=X_train.columns)
print(imp.sort_values(ascending=False).head(5).round(3))
```

`feature_importances_` sums, over every node that split on a feature, the weighted impurity reduction that split achieved — the ΔI of the previous lecture, accumulated.

It runs, it is fast, and the top of the list is entirely sensible: elevation first, then the distances. Nothing invites suspicion.

✓ Before trusting any ranking, ask what it would look like on a variable you *know* carries no information. If you cannot answer, add one and find out.

Add a control column and refit

```
rng = np.random.default_rng(42)
X_tr2 = X_train.assign(random_decoy=rng.random(len(X_train)))
X_te2 = X_test.assign(random_decoy=rng.random(len(X_test)))

rnd2 = RandomForestClassifier(n_estimators=100, max_features="sqrt",
                              random_state=42, n_jobs=-1).fit(X_tr2, y_train)
```

A column of uniform random numbers. It cannot possibly carry information about which trees grow where. Where does it rank among the 55?

The measurement with the control built in

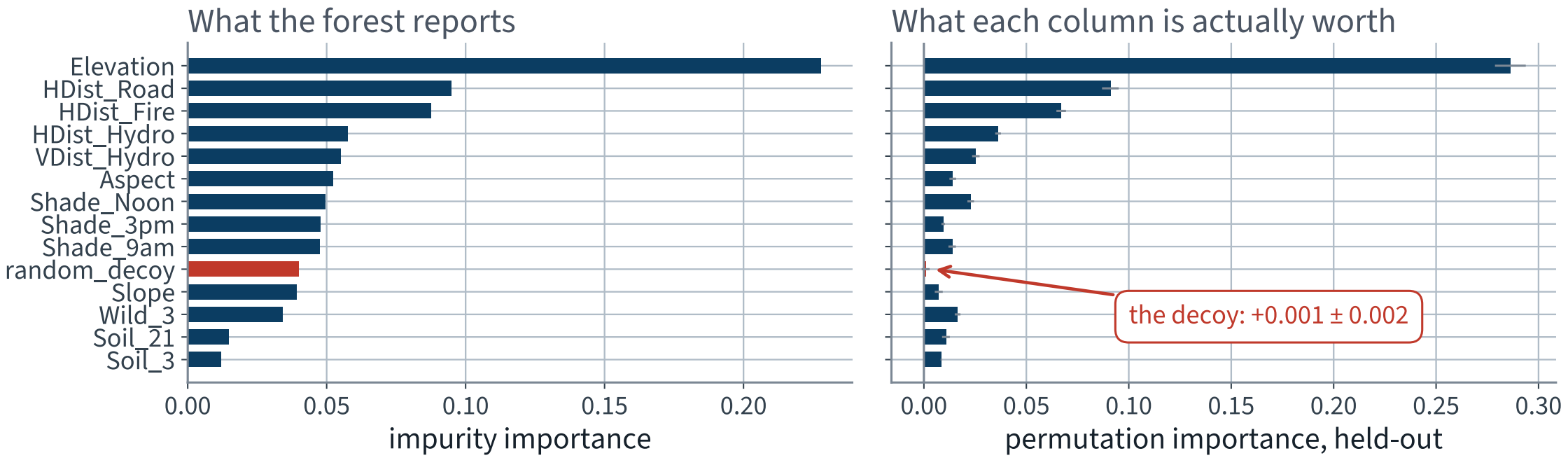
Before the answer, a second measurement to set beside it. Shuffle one column of the **held-out** set, re-score, and see how much accuracy falls. Repeat, and report a spread.

```
from sklearn.inspection import permutation_importance

r = permutation_importance(rnd2, X_te2, y_test, n_repeats=5,
                           random_state=42, n_jobs=-1)
```

> **SCOPE**
>
> `permutation_importance` goes beyond Chapter 6. It is here BEYOND THE SYLLABUS, FOR CONTEXT because the alternative is to ship the biased measurement, and because the idea — break the column and see what breaks — is worth knowing.

The same columns, measured two ways



✓ Same model, same columns, same ordering of the rows. The decoy is tenth on the left and zero on the right.

Boosting — a different idea entirely

Everything so far trains members **in parallel** and averages them. Boosting trains them **in sequence**, each one correcting its predecessor.

> The members are not identically distributed and are certainly not exchangeable, so **today's formula does not apply to boosting**. Bagging reduces variance; boosting reduces *bias*.

✗ Two consequences: boosting cannot be parallelised across members, and it is much easier to overfit with. Both follow from the sequence.

AdaBoost

1. Fit a weak learner — typically a tree of depth 1, a *decision stump*
2. **Increase the weight** of the instances it got wrong
3. Fit the next learner on the reweighted data; repeat
4. Predict by a weighted vote, each learner weighted by its own accuracy

```
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1),
                         n_estimators=200, learning_rate=0.5,
                         random_state=42)
```

If it overfits, reduce `n_estimators` or `learning_rate`, or regularise the base estimator.

Gradient boosting

Same sequence, different correction. Instead of reweighting the instances, fit each new learner to the **residual errors** of the ensemble so far.

- `learning_rate` scales each member's contribution. A low rate needs more members and generalises better — this is *shrinkage*
- Too few members underfits; too many overfits. Use early stopping with `n_iter_no_change`
- The number of members is therefore *not* a "more is better" parameter, which is exactly the opposite of bagging

> **WHEN IT BITES**
>
> Adding trees to a bagged ensemble can only help, because $(1 - \rho)\sigma^2/n$ falls monotonically. Adding trees to a boosted ensemble eventually hurts. Carrying the first intuition into the second is one of the commonest mistakes with these models.

What practitioners actually reach for

`HistGradientBoostingClassifier` bins each feature into at most 255 buckets before growing anything, so a split search costs $O(\text{bins})$ rather than $O(m)$.

```
from sklearn.ensemble import HistGradientBoostingClassifier

hgb = HistGradientBoostingClassifier(max_iter=100, learning_rate=0.05,
                                     early_stopping=False,
                                     random_state=42).fit(X_train, y_train)
```

- Dramatically faster on a dataset this size, and it is the same algorithm LightGBM and XGBoost implement
- Handles missing values and categorical columns natively — no imputer, no encoder
- On tabular data it is the usual first choice, and on most benchmarks it is the strongest single model

✗ On *this* dataset, at a budget that fits a lecture, it lands several points below the bagged forest. The notebook measures both, and measures what happens when you raise the learning rate. Usual is not always.

Stacking

Rather than voting or averaging, **train a model to do the aggregation**. The base learners' predictions become the input features of a final estimator, the *blender*.

- The blender must be trained on predictions the base learners made for instances they did not see — otherwise it is learning from memorised answers
- `StackingClassifier` does this with cross-validation internally, which is the only reason it is safe to use

> **WHEN IT BITES**
>
> Build stacking by hand and the blender sees in-sample predictions, which are far too good. It then learns to trust whichever base learner overfits hardest, and the whole ensemble is worse than its best member — with no error and no warning.

Which to reach for

Situation	Reach for
A high-variance base learner and time to spare	bagging or a random forest
Many weakly informative columns	a random forest — feature subsampling earns its keep
Training time is the binding constraint	extra-trees
You want the strongest tabular model	histogram gradient boosting — usually, and measure it against a forest anyway
You must justify individual predictions	**✗ none of them — go back to one tree**

The last row is the one this pair of lectures exists to teach, and it is the row people skip.

Did the averaging fix what we diagnosed?

The diagnosis was *instability*, not inaccuracy. So test the thing that was diagnosed, not the thing that is easy to measure.

Random forest, 20 disjoint training sets	Measured
Variance of one member's correctness	**✗ 0.16734**
Variance of the average of 20	**0.01508 ✓**
Share of the variance the averaging removed	**91.0% ✓**
The floor $\rho\sigma^2$ it cannot reach below	0.00721

✓ Nine tenths of a single member's variance, gone — and we knew before starting both that it would stop and roughly where. The notebook repeats the twenty-subsample disagreement measurement on a forest; predict the answer before you run it.

The notebook for this lecture

Open Lecture 7 in Colab

- **Runs on free CPU.** It rebuilds the previous lecture's split from the seed, so nothing has to be carried across from that notebook
- Everything on today's slides, on the same rows with the same seed: the twenty refitted trees, the four 200-member pools, the twenty disjoint training sets and ρ measured on them, the decoy column, both importance measurements and the boosted models
- It checks the formula twice — once on synthetic predictors with a ρ you choose, and once against the four measured ensembles

> **WHAT TO CHANGE BEFORE THE NEXT LECTURE**
>
> Set `max_features=None` on the random forest, which turns it back into bagging. Watch ρ rise, each member get better, and the ensemble accuracy rise with it — the trade of the whole lecture, in one keyword.

What we did

1. Started from a tree whose accuracy moves by 0.25 points under refitting while 9.1% of its predictions change: variance
2. $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/n$: averaging destroys the independent component and cannot touch the correlated one, so the way to improve an ensemble is *less correlated* members, not more of them
3. Built bagging, random forests and extra-trees, and identified what each randomises: rows, then columns, then thresholds
4. Measured ρ on 20 disjoint training sets — from 0.072 for bagging to 0.043 for the forest — with the measured variances on the curves the formula predicts
5. Found the lowest-ρ ensemble is *not* the most accurate, because every mechanism that lowers ρ also weakens the members, and σ^2 is in both terms
6. Bought about fifteen points of accuracy with 1,497,919 leaves — and lost the per-prediction justification the previous lecture was built around
7. Showed impurity importance ranking a column of pure noise 10th of 55, and repaired it with a permutation measurement on held-out data
8. Boosting trains its members in sequence and reduces bias, so the formula does not apply to it, and more members can hurt

In the book

Topic	Chapter
Voting classifiers, hard and soft	6
Bagging, pasting, out-of-bag evaluation	6
Random forests, extra-trees, feature importance	6
AdaBoost, gradient boosting, histogram-based boosting	6
Stacking and the blender	6
Decision trees, impurity, instability	5
The bias–variance decomposition	4

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 7

five, in the style of the written paper

Exercises — Lecture 7

1 For n predictors each of variance σ^2 and pairwise correlation ρ, the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$. State the limit as $n \to \infty$, and what it means for ensembles. [5]

2 Bagging measures $\rho = 0.072$ with one member at 78.8% accuracy; the random forest measures $\rho = 0.043$ with one member at 73.6%. Yet bagging's ensemble is the more accurate. Explain, using the formula above. [5]

3 You set `bootstrap=False` and `oob_score=True`. State what happens and why. [3]

Solutions on Lecture 8's deck.

Exercises — Lecture 7 (continued)

4 Impurity importance ranks a column of random numbers tenth of 55; permutation importance on held-out data puts it at zero. Describe how permutation importance is computed, and why a random column cannot score well on it. [5]

5 A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a contradiction? Answer with a reason. [4]

Solutions on Lecture 8's deck.

THE FIVE SET AT THE END OF LECTURE 6

Solutions Lecture 6

not part of this lecture

Solutions — Lecture 6

1. A node holds 100 training patches: 50 of one species, 30 of a second and 20 of a third. Compute its Gini…

✓ $G = 1 - (0.5^2 + 0.3^2 + 0.2^2) = 1 - 0.38 = 0.62$. A pure node scores 0.

- The class distribution is $(0.5, 0.3, 0.2)$; the sum of squares, 0.38, is the probability that two draws with replacement are the same species, so 0.62 is the probability that they differ
- A pure node has one $p_k = 1$, so the sum of squares is 1 and $G = 0$

2. On a two-class problem, entropy halved and Gini never differ by more than 0.055 at any node. Two candidate…

✓ No. Each split's weighted entropy halved is its weighted Gini plus a gap between 0 and 0.055, so the 0.08 difference shrinks to at least 0.025 and keeps its sign.

- Halving entropy multiplies every candidate's impurity by the same factor, so it changes no ranking
- A weighted average of gaps that each lie between 0 and 0.055 also lies between 0 and 0.055; the two criteria can disagree only when the Gini difference is smaller than that — a close call

Solutions — Lecture 6 (2)

3. An unconstrained tree with 5,699 leaves scores 100.0% on its 48,000 training patches. Explain why that number…

✓ With about eight patches per leaf the tree has isolated almost every training patch, so 100% holds by construction. Report the cross-validated accuracy on the training set — 82.1% here.

- Nothing stopped the tree growing until almost every leaf was pure, so its training accuracy measures memory, not generalisation
- Cross-validated folds score it on rows it was not fitted to; the test set is kept for the end and used once

4. Under the depth-8 cap the grid prefers min_samples_leaf=1 (73.75% cross-validated) to 20 (73.35%). Give the…

✓ A leaf of one turns the justification into ‘100% of the 1’ — a single patch, which is not the auditable evidence the brief requires. Report the 0.40 points it costs.

- The justification quotes the share of the leaf’s training patches; it is only evidence if the leaf holds enough of them
- When the brief constrains the model the grid does not get a vote — but a constraint that overrules a measurement has to show the measurement it overruled

Solutions — Lecture 6 (3)

5. Per-class recall shows one class far below the rest. Give two responses that do not involve changing the…

✓ Report per-class recall beside the headline, and take the question back to the client. Report first.

- A single accuracy averages the class away; per-class recall costs nothing and makes the failure visible, which is the only response available before anyone has decided anything
- Whether that class is worth other classes' recall is the agency's decision, not a machine learning one — and reweighting, the obvious next step, is a different model answering a different question

LECTURE 8 · GÉRON CH. 7–8

Dimensionality reduction and unsupervised learning

PCA via the SVD — in what sense the principal subspace minimises reconstruction error, and how many components to keep

Then clustering: k-means, the silhouette, and what a score cannot choose

DBSCAN, Gaussian mixtures and anomaly detection

Run the Lecture 7 notebook first